

MATLAB Cheatsheet	
Basics • Linear Algebra • Robot Dynamics (151-0851-00L, HS26)	
1 Environment & Workspace	
<code>clc</code>	<i>% clear command window</i>
<code>clear / clear x</code>	<i>% clear all / one variable</i>
<code>close all</code>	<i>% close all figures</i>
<code>clearvars -except q</code>	<i>% keep q</i>
<code>who, whos</code>	<i>% list variables (with size/class)</i>
<code>help pinv</code>	<i>% short help; doc pinv -> full docs</i>
<code>which myFun</code>	<i>% which file is called?</i>
<code>exist('x','var')</code>	<i>% 1 if variable w exists</i>
<code>pwd, cd('..'), ls</code>	<i>% folders</i>
<code>addpath(genpath('ex1'))</code>	<i>% add folder + subfolders to path</i>
<code>format long / format short / format compact</code>	
<code>x = 3;</code>	<i>% suppresses output</i>
<code>x = 3</code>	<i>% no ; -> prints w</i>
<code>...</code>	<i>% line continuation</i>
<code>% comment</code>	<i>%% section (Ctrl+Enter runs section)</i>
<code>Ctrl+C</code>	<i>% abort running code</i>
Constants: pi, Inf, NaN, eps ($2.2 \cdot 10^{-16}$), i (imag. unit). ans = last unassigned result.	

2 Creating Arrays	
<code>a = 5;</code>	<i>% scalar (1x1 double)</i>
<code>r = [1 2 3]; r = [1, 2, 3];</code>	<i>% row vector 1x3</i>
<code>c = [1; 2; 3];</code>	<i>% column vector 3x1</i>
<code>c = r';</code>	<i>% transpose -> column</i>
<code>A = [1 2; 3 4];</code>	<i>% 2x2 matrix (; = new row)</i>
<code>x = 0:0.1:1;</code>	<i>% start:step:end</i>
<code>x = 1:6;</code>	<i>% step 1</i>
<code>x = 10:-2:0;</code>	<i>% descending</i>
<code>x = linspace(0, 1, 50);</code>	<i>% 50 points incl. ends</i>
<code>zeros(3), zeros(3,6), ones(2,3), eye(4)</code>	
<code>rand(3,1)</code>	<i>% U(0,1) randn(6,1) % N(0,1)</i>
<code>randi(10,1,5)</code>	<i>% 5 ints in 1..10</i>
<code>diag([1 2 3])</code>	<i>% diagonal matrix</i>
<code>diag(A)</code>	<i>% extract diagonal (column)</i>
<code>blkdiag(A, B)</code>	<i>% block-diagonal</i>
<code>repmat(v, 1, 4)</code>	<i>% tile</i>
<code>[A, B] / horzcat(A,B)</code>	<i>% side by side</i>
<code>[A; B] / vertcat(A,B)</code>	<i>% stacked</i>
<code>cat(3, A, B)</code>	<i>% 3-D array</i>
<code>[X,Y] = meshgrid(x, y);</code>	<i>% grids for surf/contour</i>
<code>NaN(3,1), true(2), false(1,4)</code>	

3 Size & Indexing	
<code>size(A)</code>	<i>% [rows cols] size(A,1) rows</i>
<code>numel(A)</code>	<i>% # elements length(v) longest dim</i>
<code>ndims(A), isempty(A), isrow(v), iscolumn(v)</code>	
<code>A(2,3)</code>	<i>% row 2, col 3</i>
<code>A(2,:)</code>	<i>% whole row 2 A(:,end) last column</i>
<code>A(1:2, [1 3])</code>	<i>% sub-block</i>
<code>A(end, end)</code>	<i>% last element</i>
<code>A(:)</code>	<i>% all elements as column (column-major!)</i>
<code>A(5)</code>	<i>% linear index (column-wise counting)</i>
<code>A(:,2) = [];</code>	<i>% delete column 2</i>
<code>v(end+1) = 7;</code>	<i>% append</i>
<code>A(A < 0) = 0;</code>	<i>% logical indexing</i>
<code>idx = find(v > 2);</code>	<i>% indices where true</i>
<code>[r, c] = find(A == max(A(:)));</code>	
<code>k = sub2ind(size(A), r, c); [r,c] = ind2sub(size(A), k);</code>	
<code>B = reshape(A, 3, []);</code>	<i>% [] = auto size</i>
<code>v = v(:);</code>	<i>% force column</i>
<code>flip(v), fliplr(A), flipud(A), circshift(v, 1)</code>	

MATLAB stores **column-major**: A(:) stacks columns. Indices start at 1. T(1:3,1:3) = rotation, T(1:3,4) = translation.

4 Operators	
<code>+</code>	<i>% add/sub (implicit expansion: A - v')</i>
<code>*</code>	<i>% MATRIX mult, right/left divide, power</i>
<code>.* ./ ./^</code>	<i>% ELEMENT-WISE</i>
<code>'</code>	<i>% conjugate transpose (use .' for sym/cplx)</i>

<code>== ~ = < > = > =</code>	<i>% relational -> logical array</i>
<code>& ~ xor</code>	<i>% element-wise logical</i>
<code>&& </code>	<i>% short-circuit (scalars, in if/while)</i>
<code>any(v), all(v)</code>	<i>% any/all true</i>
<code>isequal(A, B)</code>	<i>% exact equality (shape + values)</i>
<code>abs(a-b) < 1e-9</code>	<i>% float comparison with tolerance</i>

A*B needs inner dims to match; A.*B needs equal (or broadcastable) sizes. A^2 = A*A but A.^2 squares entries.

5 Elementary Functions	
<code>sin cos tan asin acos atan atan2(y,x)</code>	<i>% radians!</i>
<code>sind cosd deg2rad rad2deg</code>	
<code>exp log log10 sqrt abs sign mod rem</code>	
<code>round(x,3) floor ceil fix</code>	
<code>sum(A)</code>	<i>% column sums sum(A,2) row sums</i>
<code>sum(A,'all')</code>	<i>% prod cumsum cumprod diff(v)</i>
<code>mean median std var min max</code>	
<code>[m, i] = max(v);</code>	<i>% value and index</i>
<code>max(A, [], 'all'), max(A, 0)</code>	<i>% elementwise max with 0</i>
<code>sort(v, 'descend'); [s, i] = sort(v);</code>	
<code>unique(v), ismember(3, v), numel(find(v>0))</code>	
<code>norm(v) vecnorm(A)</code>	<i>% column-wise 2-norms</i>

6 Control Flow	
<code>if x > 0 && ~isempty(v)</code>	<i>...</i>
<code>elseif x == 0</code>	<i>...</i>
<code>else</code>	<i>...</i>
<code>end</code>	
<code>for k = 1:n</code>	<i>% also: for col = A (iterates cols)</i>
<code>if k == 3, continue; end</code>	
<code>if done, break; end</code>	
<code>end</code>	
<code>while norm(e) > tol && it < maxIt</code>	<i>it = it + 1;</i>
<code>end</code>	
<code>switch mode</code>	<i>case 'pos', ...</i>
<code>case {'rot','ori'},...</code>	
<code>otherwise, ...</code>	
<code>end</code>	
<code>try</code>	<i>x = riskyFun();</i>
<code>catch err</code>	<i>disp(err.message);</i>
<code>end</code>	

7 Functions	
7.1 Function file myFun.m (name = file name)	
function [y, J] = myFun(q, k)	
<i>%MYFUN One-line description (shown by help)</i>	
<code>if nargin < 2, k = 1; end</code>	<i>% default argument</i>
<code>y = k*q;</code>	
<code>if nargin > 1, J = k*eye(numel(q)); end</code>	
<code>end</code>	

<code>function h = helper(x)</code>	<i>% local function: same file,</i>
<code>h = 2*x;</code>	<i>% after the main one, private</i>
<code>end</code>	

Scripts: plain .m without function, share the base workspace.

7.2 Anonymous functions & handles	
<code>f = @(x) x.^2 + 3*x;</code>	<i>% f(2) -> 10</i>
<code>g = @(x,y) x'*y;</code>	
<code>Rz = @(a) [cos(a) -sin(a) 0; sin(a) cos(a) 0; 0 0 1];</code>	
<code>h = @myFun;</code>	<i>% handle to file fn</i>
<code>feval('myFun', q)</code>	<i>% call by name string</i>
<code>fplot(f, [-2 2])</code>	
<i>% anonymous fns capture variables at creation time!</i>	
7.3 Variable arguments & map-style calls	
<code>function out = f(varargin)</code>	<i>% varargin{1}, ...</i>
<code>n = numel(varargin);</code>	
<code>end</code>	
<code>[a, ~] = myFun(q);</code>	<i>% ignore output</i>

<code>arrayfun(@(k) k^2, 1:5)</code>	<i>% apply to each element</i>
<code>arrayfun(@(k) Rz(k), [0 pi], 'UniformOutput', false)</code>	
<code>cellfun(@numel, {'ab','cde'})</code>	<i>% -> [2 3]</i>

8 Data Types	
<code>class(x), isa(x,'double'), isnumeric, islogical</code>	
<code>double(x), int32(x), logical(x)</code>	
<code>% --- char vs string</code>	
<code>s1 = 'text'; s2 = "text";</code>	<i>% char array vs string</i>
<code>[s1 ' more'], s2 + " more", strcat('a','b')</code>	
<code>num2str(pi,4), str2double('3.1')</code>	<i>% mat2str(A)</i>
<code>sprintf('q%d', 3)</code>	<i>% -> 'q3'</i>
<code>strcmp(a, b), contains(s, 'x'), upper(s), split(s, ',')</code>	
<code>% --- cell arrays (heterogeneous containers)</code>	
<code>c = {1, 'two'}, [3 4 5];</code>	
<code>c{3}</code>	<i>% content -> [3 4 5]</i>
<code>c(3)</code>	<i>% sub-cell -> {[3 4 5]}</i>
<code>c = cell(1,6); c{k} = T{k};</code>	<i>% e.g. store transforms</i>
<code>% --- structs</code>	
<code>p.m = 2.5; p.l = [0.3 0.2]; p.name = 'link1';</code>	
<code>p = struct('m', 2.5, 'l', 0.3);</code>	
<code>p.('m')</code>	<i>% fieldnames(p), isfield(p,'m'), rmfield(p,'m')</i>
<code>links(2).m = 1.2;</code>	<i>% struct array</i>
<code>[links.m]</code>	<i>% collect field values</i>
<code>% --- maps</code>	
<code>M = containers.Map({'a','b'}, {1, 2}); M('a')</code>	

9 Printing & Files	
<code>disp(A)</code>	<i>disp(['q = ', num2str(q')])</i>
<code>fprintf('err = %.3e, it = %d\n', err, it);</code>	
<code>fprintf('%6.3f %6.3f %6.3f\n', A'); % prints ROWS of A</code>	<i>% %d int, %f float, %4f, %e sci, %g compact, %s string</i>
<code>save('data.mat', 'q', 'T');</code>	<i>load('data.mat');</i>
<code>S = load('data.mat');</code>	<i>S.q</i>
<code>writematrix(A, 'A.csv');</code>	<i>A = readmatrix('A.csv');</i>
<code>T = readtable('log.csv');</code>	<i>T.time</i>

10 Linear Algebra	
10.1 Products & basic quantities	
<code>A*B</code>	<i>% matrix product a'*b % dot (cols)</i>
<code>dot(a,b)</code>	<i>cross(a,b) % 3-vectors</i>
<code>a*b'</code>	<i>% outer product kron(A,B)</i>
<code>A' A.'</code>	<i>% transposes A'*k % matrix power</i>
<code>trace(A), det(A), rank(A), rank(A, tol)</code>	
<code>norm(v)</code>	<i>% 2-norm norm(v,1), norm(v,Inf)</i>
<code>norm(A)</code>	<i>% spectral norm(A,'fro')</i>
<code>cond(A)</code>	<i>% condition number (large -> near singular)</i>
<code>triu(A), tril(A), diag(A), diag(diag(A))</code>	
<code>issymmetric(A), isdiag(A), istriu(A)</code>	

10.2 Solving linear systems	
<code>x = A \ b;</code>	<i>% solve A x = b (DON'T use inv(A)*b)</i>
<code>X = B / A;</code>	<i>% solve I A = B</i>
<code>x = pinv(A) * b;</code>	<i>% Moore-Penrose: least-squares + MIN-NORM</i>
<code>x = lsqminnorm(A, b);</code>	<i>% min-norm LS, faster than pinv</i>
<code>Ainv = inv(A);</code>	<i>% only if you really need the matrix</i>

A\b for a fat (underdetermined) A returns a basic solution with many zeros – **not** the minimum-norm one. For redundant robots ($J \in \mathbb{R}^{m \times n}$, $n > m$) use `pinv(J) / lsqminnorm`. For tall A, A\b = least squares.

10.3 Decompositions	
<code>e = eig(A);</code>	<i>% eigenvalues</i>
<code>[V, D] = eig(A);</code>	<i>% A*V = V*D</i>
<code>s = svd(A);</code>	<i>% singular values (descending)</i>
<code>[U,S,V] = svd(A);</code>	<i>% A = U*S*V'</i>
<code>[U,S,V] = svd(A, 'econ');</code>	<i>% economy size</i>
<code>[Q, R] = qr(A);</code>	<i>% A = Q*R</i>
<code>[L,U,P] = lu(A);</code>	<i>% P*A = L*U</i>
<code>R = chol(A);</code>	<i>% A = R'*R (A must be SPD)</i>
<code>[R, p] = chol(A);</code>	<i>% p == 0 <=> A is SPD</i>
<code>N = null(A);</code>	<i>% orthonormal basis of ker(A)</i>
<code>O = orth(A);</code>	<i>% orthonormal basis of range(A)</i>
<code>expm(A), logm(A), sqrtm(A)</code>	<i>% matrix exp/log/sqrt</i>

10.4 Useful checks	
<code>all(eig((M+M')/2) > 0)</code>	<i>% positive definite?</i>
<code>norm(C'*C - eye(3)) < 1e-9</code>	<i>% orthogonal?</i>

<code>abs(det(C) - 1) < 1e-9</code>	<i>% proper rotation?</i>
<code>min(svd(J))</code>	<i>% distance to singularity</i>
<code>sqrt(det (J'*J))</code>	<i>% manipulability</i>

11 Symbolic Math Toolbox	
<code>syms x y real</code>	<i>% scalar symbols</i>
<code>syms q [6 1] real</code>	<i>% q1..q6 as vector q</i>
<code>q = sym('q', [6 1], 'real');</code>	<i>% same, older versions</i>
<code>syms m l g positive</code>	
<code>assume(x > 0)</code>	
<code>f = x^2*sin(y);</code>	
<code>df = diff(f, x); d2f = diff(f, x, 2);</code>	
<code>J = jacobian([f; x*y], [x; y]); % Jacobian of vector fn</code>	
<code>H = hessian(f, [x; y]);</code>	
<code>F = int(f, x); F = int(f, x, 0, 1);</code>	
<code>f2 = simplify(f); expand(f), collect(f, x), factor(f)</code>	
<code>subs(f, x, 2)</code>	<i>% substitute</i>
<code>subs(f, {x, y}, {1, pi})</code>	
<code>double(subs(f, [x y], [1 2]))</code>	<i>% -> numeric</i>
<code>vpa(pi, 8), pretty(f), latex(f)</code>	
<code>sol = solve(x^2 == 4, x);</code>	<i>% symbolic solve</i>
<code>[sx, sy] = solve([x*y==3, x-y==1], [x y]);</code>	
<code>taylor(sin(x), x, 'Order', 4)</code>	
<code>A = sym(zeros(3));</code>	<i>% symbolic preallocation</i>
<code>% symbolic -> fast numeric function / m-file</code>	
<code>fh = matlabFunction(J, 'Vars', {x, y});</code>	
<code>matlabFunction(M, 'File','M_fun', 'Vars', {q}, ...</code>	
<code>'Optimize', false); % faster generation</code>	

Time derivative of a matrix A(q): dA = reshape(jacobian(A(:),q),dq, size(A)). Avoid simplify on huge expressions – it can take minutes. Use `.` (not `'`) or declare symbols `real`, `else` conjugates.

12 Numerical Methods	
<i>% ODE: da/dt = f(t,a)</i>	
<code>opts = odeset('RelTol',1e-8, 'AbsTol',1e-10);</code>	
<code>[t, X] = ode45(@(t,x) f(t,x,p), [0 5], x0, opts);</code>	
<i>% I: rows = time steps, cols = states; I(:,1) = a1(t)</i>	
<code>[t, X] = ode45(fh, 0:0.01:5, x0);</code>	<i>% fixed output grid</i>
<i>% stiff: ode15s; fixed step: write your own loop</i>	
<code>for k = 1:N, x = x + dt*f(t(k), x); end</code>	<i>% expl. Euler</i>
<code>xr = fzero(@(x) cos(x) - x, 0.5);</code>	<i>% scalar root</i>
<code>xs = fsolve(@(x) F(x), x0);</code>	<i>% nonlinear system</i>
<code>xm = fminsearch(@(x) cost(x), x0);</code>	<i>% derivative-free</i>
<code>xm = fminunc(@(cost, x0);</code>	<i>% Optim. TB</i>
<code>x = quadprog(H, f, A, B, Aeq, beq, lb, ub);</code>	<i>% QP</i>
<code>x = lsqlin(C, d, A, B, Aeq, beq);</code>	<i>% constrained LS</i>
<code>yi = interp(t, y, ti, 'linear');</code>	<i>% Optim. TB</i>
<code>I = integral(@(x) x.^2, 0, 1);</code>	
<code>trapz(t, y), cumtrapz(t, y), gradient(y, dt)</code>	
<code>pp = polyfit(x, y, 3); polyval(pp, xq); roots(pp)</code>	

12.1 Control System Toolbox	
<code>sys = ss(A, B, C, D); tf(sys), step(sys), bode(sys)</code>	
<code>K = lqr(A, B, Q, R); eig(A - B*K)</code>	
<code>K = place(A, B, poles);</code>	
<code>rank(ctrb(A, B)), rank(obsv(A, C))</code>	

13 Plotting	
<code>figure('Name','Joints');</code>	<i>hold on; grid on;</i>
<code>plot(t, q(1,:), 'b-', 'LineWidth', 1.5);</code>	
<code>plot(t, q_d(1,:), 'r--);</code>	
<code>xlabel('t [s]'); ylabel('q_1 [rad]); title('Tracking');</code>	
<code>legend('q', 'q_d', 'Location','best'); xlim([0 5]);</code>	
<code>subplot(2,1,1); ... subplot(2,1,2); ...</code>	
<code> tiledlayout(2,2); nexttile; plot(...);</code>	
<code>plot3(x, y, z); axis equal; view(3); % 3-D path</code>	
<code>scatter(x, y, 10, c, 'filled');</code>	
<code>quiver(3(x0,y0,z0, u,v,w, 0);</code>	<i>% frame azes/arrows</i>
<code>surf(X, Y, Z); shading interp; colorbar;</code>	
<code>ylime(0,'k:'); xline(1);</code>	
<code>exportgraphics(gcf, 'fig.pdf');</code>	<i>% save figure</i>
<code>% animation</code>	
<code>h = plot3(nan,nan,nan, 'o-');</code>	
<code>for k = 1:numel(t)</code>	
<code>set(h, 'XData', P(1,:), 'YData', P(2,:),k), 'ZData', P</code>	

```
(3,:k));
drawnow; % pause(0.01)
end
```

14 Robot Dynamics Toolkit

14.1 Rotations & transforms

```
skew = @(v) [0 -v(3) v(2); v(3) 0 -v(1); -v(2) v(1) 0];
unskew = @(S) [S(3,2); S(1,3); S(2,1)];
Rx = @(a) [1 0 0; 0 cos(a) -sin(a); 0 sin(a) cos(a)];
Ry = @(a) [cos(a) 0 sin(a); 0 1 0; -sin(a) 0 cos(a)];
Rz = @(a) [cos(a) -sin(a) 0; sin(a) cos(a) 0; 0 0 1];
C_IB = Rz(yaw)*Ry(pitch)*Rx(roll); % ZYX Euler
A_r = C_AB * B_r; % change frame
C_AC = C_AB * C_BC; % chain
C_BA = C_AB'; % inverse
% homogeneous transforms
T = [C, r; 0 0 0 1];
Ti = [C', -C'*r; 0 0 0 1]; % inverse
p = T * [r_B; 1]; p = p(1:3); % map point
C = T(1:3,1:3); r = T(1:3,4);
% rotation vector <-> matrix via matrix exp/log
C = expm(skew(phi)); % phi = theta*n
phi = unskew(logm(C)); % (theta < pi)
th = acos(trace(C)-1)/2); % rotation angle
% Euler angles from C (ZYX): use atan2, never atan
yaw = atan2(C(2,1), C(1,1));
pitch = atan2(-C(3,1), sqrt(C(3,2)^2 + C(3,3)^2));
roll = atan2(C(3,2), C(3,3));
wrap = @(a) atan2(sin(a), cos(a)); % to (-pi,pi]
```

14.2 Quaternions [w; x; y; z]

```
qmul = @(p,q) [p(1)*q(1) - p(2:4)'*q(2:4);
p(1)*q(2:4) + q(1)*p(2:4) + cross(p(2:4),q
(2:4))];
qconj = @(q) [q(1); -q(2:4)];
q = q / norm(q); % normalise!
q2C = @(q) (2*q(1)^2-1)*eye(3) + 2*q(1)*skew(q(2:4)) ...
+ 2*q(2:4)*q(2:4)';
```

Robotics/Navigation TB built-ins: eul2rota([z y x]) (default 'ZYX'), rota2eul, rota2quat (returns row [w x y z]), quat2rota, axang2rota([n' th]). Cross-check against your own functions – the exercises auto-grade your implementation.

14.3 Loops over a kinematic chain

```
T_I = cell(1,n); T = T_I0;
for k = 1:n
T = T * T_k(q, k); % T_{k-1,k}(q_k)
T_I{k} = T; % store T_Ik
end
r_IE = T_I{n}(1:3,4);
% geometric Jacobian, revolute joints (all in I)
J_P = zeros(3,n); J_R = zeros(3,n);
for k = 1:n
n_k = T_I{k}(1:3,1:3) * axis_k(:,k); % axis in I
J_P(:,k) = cross(n_k, r_IE - T_I{k}(1:3,4));
J_R(:,k) = n_k;
end
J = [J_P; J_R];
```

14.4 Finite-difference check of any Jacobian

```
function J = numJac(f, q, h)
if nargin < 3, h = 1e-6; end
f0 = f(q); J = zeros(numel(f0), numel(q));
for k = 1:numel(q)
e = zeros(size(q)); e(k) = h;
J(:,k) = (f(q+e) - f(q-e)) / (2*h);
end
end
norm(numJac(@jointToPosition, q) - jointToPosJac(q))
```

14.5 Pseudo-inverse & null-space

```
Jp = pinv(J); % J+
Jd = J' / (J*J' + lambda*2*eye(size(J,1))); % damped
N = eye(n) - pinv(J)*J; % projector
dq = pinv(J1)*w1 + N1*pinv(J2*N1)*(w2 - J2*pinv(J1)*w1);
```

14.6 Equations of motion

```
% H(q) d2q + b(q,dq) + g(q) = tau
ddq = M \ (tau - b - g); % forward dyn.
tau = M*ddq_des + b + g; % inverse dyn.
```

```
Lam = inv(J/M*J'); % op-space
inertia
% simulate
rhs = @(t,x) [x(n+1:end);
M_fun(x(1:n)) \ (tau_fun(t,x) - b_fun(x(1:n),x(n+1:end)
)) - g_fun(x(1:n))];
[t, X] = ode45(rhs, [0 5], [q0; dq0]);
```

15 Debugging & Performance

```
dbstop if error % break into debugger on error
keyboard % manual breakpoint in code
dbcont / dbquit % continue / leave debugger
size(x), class(x) % first thing to check on errors
assert(isequal(size(J), [6 6]), 'J has wrong size');
tic; ...; toc % timing
profile on; run; profile viewer
q = zeros(6,N); % PREALLOCATE before loops
y = A*x; % vectorise instead of loops
```

Common errors: “Dimensions of arrays being concatenated are not consistent” → row/column mix. “Incorrect dimensions for matrix multiplication” → use .* or transpose. “Index exceeds array bounds” → check size. “Undefined function” → path/addpath, file name ≠ function name.